Pasted text.txt

TXT

HW2: Multiple Linear regression
先不要直接做
分析一下以及你覺得應該怎麼做

規劃模型評估指標與視覺化呈現方式
先整理需求，再講我會怎麼做。

一、作業需求拆解

必做（基本分）

1. 資料集：來自 Kaggle，特徵數 > 13，並註明來源與連結。
2. 模型：線性回歸，可以是簡單、多元或 Auto Regression。重點放在多元線性回歸。
3. 一定要有：特徵選擇（Feature Selection）和模型評估（Model Evaluation）。
4. 預測圖要附信賴區間或預測區間。這點很容易被忽略。
5. 報告要照 CRISP-DM 六個階段寫。
6. AI 相關：
   - ChatGPT 對話匯出成 PDF
   - 用 NotebookLM 研究網路上的同題解法，寫 100 字以上摘要
   - 報告裡要明確標出「GPT 輔助內容」和「NotebookLM 摘要」
7. 繳交：GitHub repo 加網頁，形式跟 DIC2 一樣。

加分

- +1：網頁有下拉選單，可以切換「兩種 problem」
- +2：可以輸入任意 Dataset URL（例如 50_Startups.csv）並直接跑分析

二、幾個要先想清楚的地方

1. 加分項決定了網頁架構，要最先決定

+2 的要求是「輸入任意 CSV URL 就能跑回歸」。所以網頁不能只放靜態報告，必須能在使用者端或伺服器端即時計算。有三條路：

方案	優點	缺點
A. Streamlit（部署到 Streamlit Cloud）	全部用 Python，沿用 statsmodels/sklearn，做下拉選單和 URL 輸入最省事	不是 GitHub Pages，要另外部署（可以從 Pages 放連結過去）
B. GitHub Pages + 純 JS（PapaParse + 矩陣運算 + Plotly）	純靜態、速度快	要用 JS 重寫 OLS 和區間計算，跟 Python 分析可能算出不一樣的結果
C. GitHub Pages + Pyodide（瀏覽器內跑 Python）	跟 notebook 共用同一份 Python 程式碼	第一次載入要等十幾秒

我的建議：如果 DIC2 是用 GitHub Pages 交的，就選 C，一份 Python 程式碼可以同時給報告和網頁用。如果 DIC2 允許 Streamlit，選 A 最省力。這點要看你 DIC2 是怎麼交的。

2.「兩種 problem」的意思不明確

可能的解讀有三種：

- (a) 簡單線性回歸（HW1）vs. 多元線性回歸（HW2）
- (b) 兩個不同的資料集
- (c) 回歸 vs. 分類

我猜是 (a) 或 (b)。建議先問老師或同學。如果問不到，下拉選單可以設計成「HW1 簡單線性回歸 / HW2 Kaggle 多元回歸 / 自訂 URL」，三種解讀大多都能涵蓋。

3. 任意 URL 的功能要夠通用

範例的 50_Startups 只有 4 個特徵，其中 `State` 是類別欄位。所以程式要能處理：

- 自動判斷數值欄位和類別欄位，類別欄位做 one-hot（drop_first，避免完全共線）
- 讓使用者用下拉選單選 target 欄位（預設最後一欄）
- 處理缺值（先 drop 或補值）
- Kaggle 的下載連結需要登入，網頁抓不到。只有 raw.githubusercontent 這類直連 CSV 才行，網頁上要寫清楚這個限制。

4. 你之前那份資料集推薦有問題

- Predict the Flood（Playground S4E5）不太適合：
  - 它是合成資料，20 個特徵的貢獻幾乎一樣，彼此也幾乎不相關，特徵選擇做出來會沒有東西可講。
  - 資料量大約百萬筆，放進網頁或 Pyodide 會很吃力。
- Boston Housing 剛好 13 個特徵，不符合「> 13」，同學很容易踩到。
- 我比較推薦有真實共線性、缺值、類別欄位的資料，特徵選擇和診斷才有內容可寫：

資料集	約略特徵數	推薦理由
Life Expectancy (WHO)	~20	有缺值、有明顯共線性（例如 infant deaths 和 under-five deaths），VIF 分析很好發揮，約 2,900 筆
King County House Sales	~18	經典房價題，約 2 萬筆，殘差異質性明顯，適合討論取 log
Car Price (CarPrice_Assignment)	~24	類別欄位多，205 筆很輕量，適合放網頁

特徵數我是憑印象估的，選定後要實際讀 CSV 確認。我首選 **Life Expectancy**，報告最有故事可寫。

5. 多元回歸的「預測圖 + 區間」怎麼畫

多元回歸沒辦法直接畫成一條線，建議畫兩張圖：

1. 測試集預測圖：x 軸是樣本（依預測值排序），畫出實際值和預測值，再加上 95% 預測區間帶。區間用 statsmodels 的 `get_prediction().summary_frame()` 取 `obs_ci_lower/upper`。
2. 單一特徵的偏效應圖：挑最重要的一個特徵，其他特徵固定在平均值，畫回歸線，再加上 CI（平均反應）和 PI（個別預測）兩條帶。

報告裡要說明 CI 和 PI 的差別，這點很加分。

三、建議流程（CRISP-DM 對應）

1. **Business Understanding**：定義要預測什麼，以及預測結果能拿來做什麼決策。
2. **Data Understanding**：
   - 看資料的 shape、型別、缺值、描述統計
   - 畫 target 分布和相關係數熱圖
3. **Data Preparation**：
   - 處理缺值，類別欄位做 one-hot，target 視情況取 log
   - 先切 train/test（例如 80/20）
   - 所有前處理和特徵選擇都只能用 **train** 資料 **fit**，避免資料洩漏
4. **Modeling**：
   - Baseline：全特徵 OLS
   - 特徵選擇比較三種方法：
     - 用 VIF 移除共線特徵
     - RFE 或 SequentialFeatureSelector，搭配 CV 選出 k
     - Lasso（係數變成 0 的特徵就是被淘汰的）
   - 最後用 statsmodels 做 OLS，取得 p-value 和區間
5. **Evaluation**：
   - 指標：RMSE、MAE、R^2、Adjusted R^2，比較 baseline 和選特徵後的模型
   - 殘差診斷：殘差 vs. 預測值、Q-Q plot（檢查線性、常態、同質變異）
   - 加上前面說的兩張預測區間圖

6. **Deployment**：網頁（下拉選單 + URL 輸入），加上 GitHub repo。

四、Repo 結構建議

```
HW2-MLR/
├── README.md              ← 報告主體（或 docs/index.html）
├── notebook/analysis.ipynb
├── src/mlr_core.py        ← 共用的回歸函式（notebook 和網頁都用這份）
├── docs/                  ← GitHub Pages 網頁
├── data/                  ← 資料集（或只放下載說明）
└── ai_logs/               ← GPT 對話 PDF、NotebookLM 摘要
```

五、需要你自己做的部分

- **NotebookLM** 摘要：把同一個資料集的 3 ~ 5 篇 Kaggle notebook 或部落格丟進 NotebookLM，請它整理大家的做法，再寫成 100 字以上的摘要。
- 對話 **PDF**：作業指定的是 ChatGPT。如果你要用跟我的對話代替，最好先跟老師確認可不可以。

另外提醒一下：這次要交 GitHub，建議一開始就建 **repo**，每完成一個 CRISP-DM 階段就 commit 一次（例如「Data preparation done」）。這樣過程有紀錄，也可以避免改壞了救不回來。之後實作時我會提醒你每個階段該 commit 什麼。

開始之前想先確認幾件事：

1. **DIC2** 是用什麼方式交的？純 GitHub Pages、Streamlit，還是其他方式？
2. 「兩種 **problem**」老師有沒有說明？
3. 資料集要用 **Life Expectancy** 嗎？還是你有其他偏好的主題？
4. 繳交期限是什麼時候？我會依時間調整範圍，例如加分項要不要全部做。

1. 純 GitHub Pages
2. 應該事跟DIC2的下拉選單一樣
3. 好
4. 10/28

Searched the web, searched past chats, and 11 more steps ▸

No connected folder on your computer, so I'll build in the workspace and send you the files at the end. The data is in; next, a Python reference analysis to see what the results look like.

Updated tasks, ran a command ▸

Car Price is noisy in its raw form: only 205 rows, and 43 columns after one-hot. Let me check whether log(price) plus the 1-SE rule for choosing k makes it more stable.

Edited 2 files, created 3 files, and 22 more steps ▸ +4

網頁和 repo 都做好了，架構跟 DIC2 一樣，是單一 `index.html`。上方下拉選單可切換 **Problem 1 · Life Expectancy**、**Problem 2 · Car Price** 和自訂 **Dataset**，自訂 Dataset 可以貼 URL 或上傳 CSV，兩個加分項都有。

結果

Problem	最終模型	特徵數	Test R^2	95% PI 覆蓋率
Life Expectancy	RFE	20 → 8	0.828	93.3%
Car Price（log 價格）	Forward SFS	43 → 8	0.854	85.4%

作業要求的預測圖有兩張，都有畫 CI（深色帶）和 PI（淺色帶）：

- 測試集預測圖
- 單一特徵偏效應圖

網頁上的數字都和 Python statsmodels 對照過，誤差小於 1e-12。三個選項我都用 headless 瀏覽器實際跑過，沒有錯誤。

你需要自己補的三件事

1. **NotebookLM 摘要（必做）**：把同主題的 Kaggle notebook 或網路文章丟進 NotebookLM，寫 100 字以上的摘要，貼到兩個地方：
   - `index.html` 搜尋「待補」，替換那一行，並照註解把 `class="warn"` 拿掉
   - `README.md` 的「NotebookLM 摘要」
2. **AI 對話 PDF**：把這段對話匯出成 PDF，放進 `ai_logs/`。作業寫的是 ChatGPT，頁面上我已誠實標註使用的是 Claude，如果擔心可以問一下老師。
3. **README 第一行的網址**：部署完成後改成實際的 GitHub Pages 網址。

上傳步驟（跟 DIC2 一樣）

1. GitHub 新建一個 **Public** repo，例如 `AIoT-HW2-MLR`。
2. 把 zip 解壓縮，按 **Add file → Upload files**，把裡面所有檔案和資料夾一起拖進去，然後 Commit。
3. 到 **Settings → Pages**，選 `main` / `/ (root)`，按 Save。

📌 Git 提醒：

- 這次不要沿用 DIC2 的 repo，要開一個新的。如果覆蓋 DIC2 的檔案，已經交出去的 DIC2 網址會壞掉。
- 之後補 NotebookLM 摘要時，直接在 GitHub 網頁上編輯 `index.html` 再 commit，網頁會自動重新部署。

老師可能會問的點

- 為什麼用 **1-SE rule**：CV 誤差在 k 很大時幾乎是平的，1-SE rule 會挑「表現差不多但特徵最少」的模型，比較不容易過擬合。所以自訂 Dataset 跑 50_Startups 會選 k=1，DIC2 選的是 k=2，兩者不衝突，可以解釋。

- **infant deaths** 和 **under-five deaths** 係數一正一負：兩者相關係數 0.997，是共線性造成的，頁面上已經有說明。
- **Car Price** 只取 8 個特徵，**Test R^2** 比全特徵低一點：測試集只有 41 筆，波動很大。選 k 要看的是 CV 結果，不是測試集。

Hw2 mlr
ZIP

Index
Code · HTML

Download all

NotebookLM 摘要（必做）：把同主題的 Kaggle notebook 或網路文章丟進 NotebookLM...

這部分可以詳細解釋需要做甚麼嗎?

整理数据集案例与学习资源建议。
作業規定要用 NotebookLM 研究「網路上同主題的解法」，寫成 100 字以上的摘要放進報告，並標示「NotebookLM 摘要」。網頁上已經留好標示和位置，只差內容。我沒辦法替你操作 NotebookLM，下面是完整步驟。

步驟 1：建立筆記本

1. 打開 https://notebooklm.google.com，用 Google 帳號登入。
2. 按「新建筆記本」，名稱可以取「HW2 Life Expectancy 回歸」。

步驟 2：加入 4～6 個來源

NotebookLM 只會根據你給的來源回答，所以來源要是別人對同一個資料集做的回歸分析。以主要題目 Life Expectancy 為例，可以用這些：

- Vipul97/life-expectancy-prediction：我們網頁內嵌的資料就是從這裡取的
- Akshu577/life-expectancy-analysis：回歸加特徵選擇
- RheaDsouza/Life-Expectancy-Prediction_World-Health-Organization：比較多種回歸模型
- KAR-NG/Life-Expectancy-Statistical-Analysis-WHO-：統計分析
- Kaggle 資料集的 Code 分頁：https://www.kaggle.com/datasets/kumarajarshi/life-expectancy-who/code，挑 1～2 篇票數高的 notebook

加來源的方法：

- 按「新增來源」，選「網站」，貼上網址。
- GitHub 網址如果讀不到內容，改貼 repo 裡的 README，或 `.ipynb` 檔案頁面的網址。
- Kaggle notebook 有時需要登入，NotebookLM 會讀不到。這時可以在 Kaggle 頁面全選複製文字，用「複製的文字」來源貼上；或用瀏覽器「列印 → 存成 PDF」後上傳 PDF。

步驟 3：請 NotebookLM 整理

在右側對話框貼這段提示：

```
請比較這些來源如何用回歸分析預測 WHO Life Expectancy 資料集，整理：
1. 缺值怎麼處理（刪除或補值）
2. 用了哪些特徵選擇方法、最後保留哪些重要特徵
3. 用了哪些模型，評估指標與結果（R²、RMSE）
4. 共同的發現（例如 HIV/AIDS、Schooling、Adult Mortality 的影響）以及常見的問題（例如共線性）
請用繁體中文，約 200 字。
```

步驟 4：改寫成你的摘要

NotebookLM 的回答不要整段照貼，建議稍微整理成自己的話，最後再加一兩句和你的作法比較，例如：

> 與上述作法相比，本作業以 5-fold CV 搭配 1-SE rule 選擇特徵數，最後保留 8 個特徵，測試集 R^2 為 0.828，與多數來源的線性回歸結果相近；同時也觀察到 infant deaths 與 under-five deaths 高度共線的問題。

這樣整段會超過 100 字，也看得出你有讀懂。建議把 NotebookLM 的畫面截圖，連同對話 PDF 一起放進 `ai_logs/`，當作使用證明。

Car Price 是第二題，不一定要另外做。想做的話，用同樣方法另開一本筆記本，來源用 https://www.kaggle.com/datasets/hellbuoy/car-price-prediction/code 。

步驟 5：貼到兩個地方

可以直接在 GitHub 網頁上改：點檔案 → 右上角鉛筆圖示 → 修改 → Commit changes。

① `index.html`：按 Ctrl+F 搜尋「待補」，會找到這兩行：

```html
<!-- 🖍️ 把 NotebookLM 摘要（100 字以上）貼在下面這個 <p> 裡，並把 class="warn" 拿
<p class="warn">（待補：請貼上 NotebookLM 對網路上同主題解法的研究摘要，100 字以上）
```

把第二行換成你的摘要（註解那行可以刪掉）：

```html
<p>NotebookLM 整理了 5 個以 WHO Life Expectancy 資料做回歸分析的專案……（你的摘要）
```

`class="warn"` 要拿掉，否則摘要會顯示成橘色的警告框。

② `README.md`：找到這一行：

```
> （待補：NotebookLM 對網路上同主題解法的研究摘要，100 字以上）
```

換成 `> 你的摘要`。摘要如果有好幾段，每段開頭都要加 `>`。

Commit 後等 1～2 分鐘，重新整理 GitHub Pages 網址，確認「AI 協助說明」區的 NotebookLM 摘要已經出現，而且不是橘色框。

如果 HTML 怕改錯，可以把 NotebookLM 的回答直接貼給我，我幫你整理成摘要，並改好 `index.html` 和 `README.md` 給你重新上傳。

problem 2 也寫一個摘要好了

Clarifying what the second problem summary should include.
我先把網頁和 README 改成有兩個摘要欄位（Problem 1 和 Problem 2 各一個），再整理 Car Price 要丟進 NotebookLM 的來源。

Edited a file, shared 3 files, and 5 more steps ▸
網頁和 README 的「NotebookLM 摘要」現在分成 Problem 1、Problem 2 兩個欄位，各有一個「待補」格子。改完後我重新測過三個選項，都正常。

摘要內容要你在 NotebookLM 裡做出來，我不能代寫。作業要求的就是 NotebookLM 的研究摘要，我寫的東西貼上去標成「NotebookLM 摘要」，就變成標示不實了。

Problem 2 的做法

1. 另開一本筆記本，例如命名為「HW2 Car Price 回歸」。不要跟 Problem 1 混在同一本，不然它會把兩個資料集的內容混在一起回答。

2. 加入來源（4～6 個）

- Kaggle 資料集的 Code 分頁：https://www.kaggle.com/datasets/hellbuoy/car-price-prediction/code ，挑 2～3 篇票數高、標題有 Linear Regression 的 notebook。這個資料集是很經典的作業題，大多數人會用 RFE 搭配 statsmodels 的 VIF 篩特徵，正好可以跟我們的做法比較。
- GitHub 上幾個用同一份資料的專案：
  - SampattKumar/Linear-Regression-Car-Dataset，我們網頁內嵌的資料就是從這裡來的
  - AayushiWani/CarPricePrediction
  - jainnnnn/Car-Price-Prediction-Model
- 加入前先看一下內容，確認用的是 `CarPrice_Assignment.csv`，也就是 205 筆、有 `CarName` 欄位的那份。Kaggle 上另外有二手車價格的資料集，名稱很像，但不是同一份。
- 網址讀不到的話，跟上次一樣：複製頁面文字貼上，或印成 PDF 再上傳。

3. 貼這段提示

```
請比較這些來源如何用線性回歸預測 CarPrice_Assignment（Geely Auto 美國市場車價）資料集，整理：
1. 前處理：CarName 怎麼處理（例如抽出品牌）、類別欄位怎麼編碼、price 有沒有取 log 或做縮放
2. 特徵選擇方法（RFE、VIF、p-value 逐步刪除等）以及最後保留的關鍵特徵
3. 模型的評估結果（R²、Adjusted R²、RMSE）
4. 共同的發現與常見問題（例如 enginesize、curbweight、horsepower 之間的共線性）
請用繁體中文，約 200 字。
```

4. 最後加一句跟我們做法的比較，例如：

> 與上述作法相比，本作業對 log(price) 建模，以 5-fold CV 搭配 1-SE rule 從 43 個特徵中選出 8 個，測試集 R^2 為 0.854；不同的是，本作業排除了 CarName，沒有抽出品牌。

最後一點值得寫：很多 Kaggle 解法會從 `CarName` 抽出品牌，品牌對車價影響很大，這是我們模型沒有用到的資訊。可以寫成「本作業的限制」或「未來可改進」。

貼到哪裡

- `index.html`：搜尋 `NLM-2`，把註解下一行的 `<p class="warn">…</p>` 換成 `<p>你的摘要</p>`。Problem 1 的摘要放在 `NLM-1`，改法一樣。
- `README.md`：搜尋「Problem 2 的 NotebookLM 摘要」，把那一行換成 `> 你的摘要`。

📌 Git 提醒：如果 repo 已經上傳過，不用重傳整包 zip，用 **Add file → Upload files** 上傳新的 `index.html` 和 `README.md`，就會覆蓋舊檔並自動重新部署。如果你已經在 GitHub 上填了 Problem 1 的摘要，覆蓋前先把它複製下來，覆蓋後再貼回 `NLM-1`，不然會被新檔蓋掉。

10/28 截止。兩份 NotebookLM 的回答做好後，可以直接貼給我，我幫你整理文字並放進兩個檔案。

Hw2 mlr
ZIP

Index
Code · HTML

Readme
Document · MD

Download all

我 problem 1 和 2 的作法是哪一個

兩題用的是同一套流程：四種方法都跑，再用 CV 自動挑出最好的那個。最後 Problem 1 選中的是 **RFE**，Problem 2 選中的是 **Forward SFS**。

共同流程（兩題都一樣）

1. 前處理：排除 ID 和類別太多的文字欄位，有缺值的列整列刪除，類別欄位做 one-hot（drop first）。
2. 切分：80% 訓練、20% 測試（seed = 42）。
3. 特徵排序：用四種方法分別排出特徵的重要順序：Forward SFS、RFE、Lasso、ElasticNet。
4. 選 k：每種方法都取前 k 個特徵做 OLS，用訓練集的 **5-fold CV RMSE** 評分，再用 **1-SE rule** 決定 k：在「CV 誤差不超過最小值 + 1 個標準誤」的範圍內，取特徵最少的 k。
5. 選方法：四種方法中，CV 誤差最低的那個就是最終模型。
6. 評估：在測試集上算 RMSE、MAE、R^2，畫 95% CI / PI 預測圖和殘差診斷。

兩題的差別

	Problem 1 · Life Expectancy	**Problem 2 · Car Price**
Target	Life expectancy（不轉換）	**log(price)**，因為價格右偏
排除欄位	Country（193 類）	car_ID、CarName（147 類）
缺值	2,938 → 1,649 筆	沒有缺值
特徵數	20	43（類別多，one-hot 後變多）
最終方法	**RFE**	**Forward SFS**
選入特徵	8 個	8 個
Test R^2	0.828	0.854

兩個方法的意思（老師問時可以這樣說）

- **RFE**（遞迴特徵消除）：先用全部特徵擬合，把標準化係數絕對值最小的那個特徵刪掉，重新擬合，再刪一個，一直重複。最後被刪的就是最重要的。
- **Forward SFS**（前向逐步選擇）：從 0 個特徵開始，每一輪試加入每一個還沒選的特徵，留下讓 CV 誤差降最多的那個，逐步加上去。

為什麼兩題結果不同：Life Expectancy 資料多（1,300 多筆訓練資料），RFE 依係數大小排序就很穩定。Car Price 只有 164 筆訓練資料，one-hot 後的虛擬變數又很多，直接用 CV 誤差挑特徵的 Forward SFS 表現比較好。這個結果是程式比較四種方法後自動選出來的，不是事先決定的。

寫 NotebookLM 摘要時的比較句

- **Problem 1**：「本作業比較四種特徵選擇方法，以 5-fold CV 搭配 1-SE rule 選出 RFE、保留 8 個特徵，測試集 R^2 為 0.828。」
- **Problem 2**：「本作業對 log(price) 建模，比較四種方法後由 Forward SFS 從 43 個特徵中選出 8 個，測試集 R^2 為 0.854。」

Problem 1：WHO 預期壽命
我透過 NotebookLM 比較了四份以 WHO 預期壽命資料集進行回歸分析的解法。
缺失值處理：各解法的做法不同。有的先刪除沒有預期壽命數值的樣本，再用該國或全體資料的中位數補齊其他欄位 [1][2]；也有的改用裝袋樹（Bagged Trees）模型推估缺失值 [3]。
特徵選擇：常見的工具包括前向 / 後向逐步回歸、相關係數矩陣、VIF 檢定，以及 PCA 與 Lasso 正則化 [1]。篩選之後，多數分析都保留了教育年限（Schooling）、HIV/AIDS、成人死亡率（Adult Mortality）、收入資源組合指數（Income composition of resources）和疫苗接種率這幾個關鍵變數 [6]。
模型與評估：除了多元線性回歸，也有解法拿 Ridge、Lasso、KNN、SVM 和集成模型（Random Forest、Boosting）來比較 [4]。線性模型多以 R^2 和 RMSE 評估，R^2 約在 0.82 到 0.86 之間，RMSE 約 3.6 到 3.7 歲 [5]。
共同發現：結論相當一致。HIV/AIDS 與成人死亡率越高，預期壽命越短；教育年限與經濟條件則與壽命呈強烈正相關 [6]。共同的難題是多重共線性，例如不同死亡率指標之間、不同經濟變數之間高度相關，通常要靠 VIF 篩選或刪除重複變數來處理 [1]。
與本作業的比較：這些解法多半用 VIF 或逐步回歸決定保留哪些變數，但「要留幾個」常靠經驗判斷。本作業同樣比較了四種特徵選擇方法，再以 5-fold 交叉驗證搭配 1-SE rule 選出 RFE、保留 8 個特徵：在誤差與最佳值相差不到一個標準誤的模型中，挑特徵最少的一個。這讓特徵數的決定有客觀依據。測試集 R^2 為 0.828，與上述線性模型 0.82–0.86 的表現相當。
Problem 2：Car Price
我透過 NotebookLM 比較了 Kaggle 上幾份以 CarPrice_Assignment 資料集、用線性迴歸預測車價的解法。
前處理：對 `CarName` 的看法分成兩派。jainnnnn、hellbuoy 與 SampattKumar 取車名第一個字當作品牌，並修正拼字錯誤（例如 maxda 改成 mazda）；AayushiWani 則視它為識別用的名稱，直接刪除。類別變數的編碼有 One-Hot Encoding、LabelEncoder，或把有順序的類別對應成數字。目標變數方面，jainnnnn 用 `log1p` 修正價格的右偏，並以 StandardScaler 標準化數值特徵；AayushiWani 則保留原始價格，也沒有做縮放。
特徵選擇：hellbuoy 與 SampattKumar 先用 RFE 粗篩，再依 VIF 與 p-value 逐一剔除共線性高或不顯著的變數；AayushiWani 依相關係數手動刪掉資訊重複的欄位，例如保留 `highwaympg`、去掉與它高度相關的 `citympg`；jainnnnn 沒有另外篩選，24 個特徵全部保留。
模型表現：AayushiWani 在原始價格上得到 R^2 約 0.81、RMSE 約 3,383 美元；jainnnnn 在 log 價格尺度上得到 R^2 0.918、RMSE 0.149。
共同發現：引擎大小、車重、馬力等規格彼此高度相關，油耗相關變數之間也一樣，多重共線性是各解法都要處理的問題。另外，車價明顯右偏，取對數後模型擬合會明顯改善。
與本作業的比較：本作業和 jainnnnn 一樣對 log(price) 建模，但多做了特徵選擇。比較四種方法後採用 Forward SFS，從 43 個特徵中選出 8 個，測試集 R^2 為 0.854。和保留全部特徵的做法相比，本作業只用不到五分之一的變數就達到相近的解釋力，也較不受多重共線性影響。

Deciding how to handle citation markers without a source list ▸
我把兩份摘要放進網頁和 README。摘要裡的 [1] ~ [6] 引用編號沒有附對應的來源清單，留著讀者也查不到，所以先拿掉。

Shared 3 files, read 2 files, ran 3 commands ▸

兩份摘要都放進網頁和 README 了。我在瀏覽器實際看過，「AI 協助說明」區會依序顯示 Problem 1、Problem 2 兩段，三個選項也都重新測過，運作正常。

我改的地方：只拿掉了 [1]~[6] 引用編號，因為頁面上沒有對應的來源清單，讀者查不到。其他文字都沒動。如果你想保留引用，把 NotebookLM 的來源清單給我，我在摘要下方補上編號對照。

Problem 2 有一處建議你再看一下：摘要寫 jainnnnn 在 log 尺度的 R^2 是 0.918，我們的 0.854 卻是轉回原始價格後算的，兩個數字不能直接比。我另外算了我們在 log 尺度的成績：

	Test R^2（log 尺度）	RMSE（log 尺度）
jainnnnn（24 個特徵，有用品牌）	0.918	0.149
本作業（8 個特徵，沒用品牌）	0.812	0.190

用同一個尺度比，我們其實比 jainnnnn 低。差距的主因應該是他們從 `CarName` 抽出了品牌，我們則直接排除這個欄位。

現在「只用不到五分之一的變數就達到相近的解釋力」這句，跟我們自己的全特徵模型比是成立的（0.854 對 0.873）。但它接在 jainnnnn 後面，容易被讀成「跟 jainnnnn 差不多」。被老師追問的話，這裡比較難解釋。建議把最後一段改成：

> **與本作業的比較：**本作業和 jainnnnn 一樣對 log(price) 建模，但多做了特徵選擇。比較四種方法後採用 Forward SFS，從 43 個特徵中選出 8 個，測試集 R^2 在原始價格尺度為 0.854、log 尺度為 0.812，略低於 jainnnnn 的 0.918。主要差異在於本作業直接排除 `CarName`，沒有像其他解法一樣抽出品牌；品牌是影響車價的重要因素，這是本作業可以改進的地方。不過，與本作業自己的全特徵模型（R^2 0.873）相比，只用不到五分之一的變數就達到相近的解釋力，也較不受多重共線性影響。

要用這個版本的話跟我說，我直接改好檔案給你。

📌 **Git 提醒：**到 GitHub repo 按 **Add file → Upload files**，上傳新的 `index.html` 和 `README.md` 覆蓋舊檔。如果還沒上傳過，就把 zip 解壓縮後整包上傳。最後記得把對話 PDF 放進 `ai_logs/`，README 第一行的網址也要改成你的 GitHub Pages 網址。

Index
Code · HTML

Readme
Document · MD

Hw2 mlr
ZIP

Download all

直接改吧

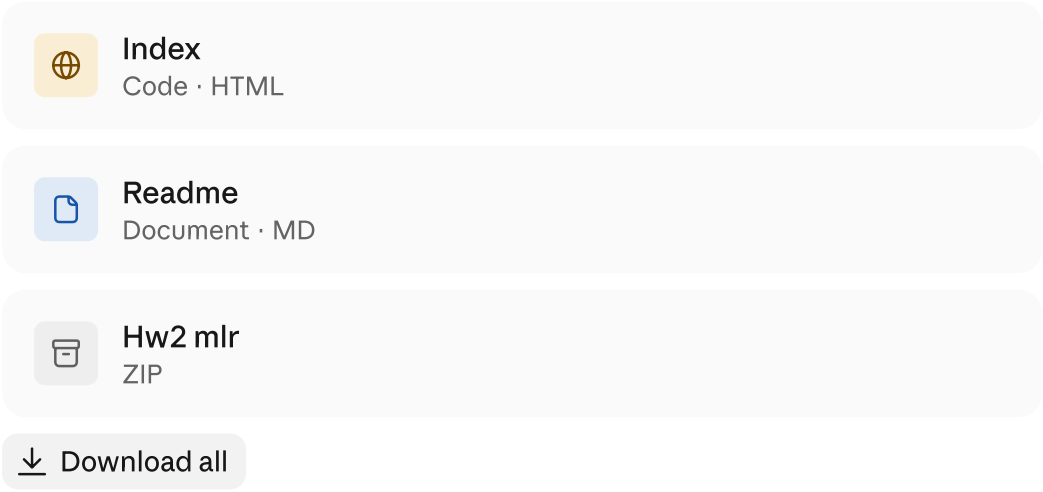
Shared 3 files, ran a command ▸

Problem 2 的「與本作業的比較」改好了，網頁和 README 用的是同一段新文字。改完後三個下拉選項我都測過，沒有錯誤。

上傳時，如果 repo 已經存在，用 **Add file → Upload files** 上傳新的 `index.html` 和 `README.md` 覆蓋舊檔就好。如果還沒上傳過，就把 zip 解壓縮後整包上傳。

10/28 交之前還剩兩件事要你自己處理：

- 把對話匯出成 PDF，放進 `ai_logs/`。如果有 NotebookLM 的截圖，也一起放進去。
- 部署完成後，把 README 第一行的網址改成你實際的 GitHub Pages 網址。



github page 是不是要重新佈署

不用手動重新部署。只要 Pages 設定的是 **Deploy from a branch**（`main` / `/ (root)`），每次 commit，GitHub 都會自動重新部署。上傳新檔案並按下 Commit changes，就等於觸發了一次部署。

確認有沒有部署成功：

1. 到 repo 上方的 **Actions** 分頁，會看到一筆 **pages build and deployment**。
   - 黃色圓圈：還在跑，通常 1～2 分鐘
   - 綠色勾勾：部署完成
   - 紅色叉叉：失敗，把錯誤畫面截圖給我
2. 打開 GitHub Pages 網址，按 **Ctrl + F5** 強制重新整理。瀏覽器可能留著舊版快取，只按 F5 有時還會看到舊內容。
3. 捲到最下面的「AI 協助說明」，看到兩段 NotebookLM 摘要，而且 Problem 2 的最後一段有「log 尺度為 0.812」，就代表已經是新版了。

只有一種情況要手動設定：這個 repo 第一次上傳，還沒開過 Pages。這時要到 **Settings → Pages**，Source 選 **Deploy from a branch**，Branch 選 `main`、資料夾選 `/ (root)`，按 Save，之後的 commit 就都會自動部署了。

📌 改 README 第一行的網址也是一次 commit，一樣會自動重新部署，不用另外處理。

Pasted text.txt

TXT

有改到嗎
網站沒顯示

Ran 3 commands

你傳來的這個 `index.html` 是新版沒錯，內容跟我最後給你的那份完全一樣。差別只在換行字元（Windows 存檔時把換行改成 CRLF），這不影響網頁顯示。所以問題不在檔案本身，而是網站上跑的還不是這份。

最常見的原因有這幾個，可以照順序檢查：

1. 檔案只改在電腦上，還沒上傳到 GitHub
   到 repo 首頁點開 `index.html`，按 Ctrl+F 搜尋「0.812」。找不到的話，代表 GitHub 上還是舊檔，要用 **Add file → Upload files** 上傳，並按下 **Commit changes**。
2. 上傳到錯的位置
   `index.html` 要放在 repo 的最外層，跟 `README.md` 同一層。如果整個解壓縮後的資料夾被拖進去，檔案會變成 `HW2-MLR/index.html`，網站讀不到。也要確認上傳的是 HW2 的 repo，不是 DIC2 的。
3. 還在部署
   到 **Actions** 分頁看最上面那一筆。黃色圓圈代表還在部署，等它變成綠色勾勾就好。
4. 瀏覽器快取
   在網站上按 **Ctrl + F5**，或用無痕視窗開網址。
5. 網址開錯
   網址要是 `https://你的帳號.github.io/HW2的repo名稱/`，不要開成 DIC2 的網址。

如果都檢查過還是不行，把 **GitHub Pages** 網址或 **repo** 網址貼給我，我直接連上去看網站現在顯示的是哪個版本、問題出在哪裡。